

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»**

ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

Кафедра Систем информатики

Направление подготовки 09.06.01 – Информатика и вычислительная техника

ОТЧЕТ

Обучающегося Козубенко Андрея Алексеевича группы № 22215 курса 4
(Ф.И.О. полностью)

Тема задания: Разработка алгоритма для реализации метода Rag

Новосибирск 2025

Содержание

1. Введение	3
2. Листинги	4
Листинг 1 - llm.py	4
Листинг 2 – Загрузка онтологии и извлечение узлов и связей	6
Листинг 3 – Преобразование узла онтологии в текст	6
Листинг 4 – Построение индекса онтологии	6
Листинг 5 – Вычисление текстовых эмбеддингов.....	7
Листинг 6 – Семантический поиск по онтологии.....	7
Листинг 7 – Основной RAG-конвейер	7
Листинг 8 – Извлечение текстовых атрибутов RDF.....	8
3. Источники информации	9

1. Введение

(Retrieval-Augmented Generation, RAG) — это подход, при котором генерация ответа большой языковой модели (LLM) осуществляется на основе данных, полученных в результате поиска во внешних источниках (файлы, базы данных, Интернет и другие источники). RAG-система работает в два основных этапа: сначала происходит извлечение релевантных документов или их частей из внешней базы знаний на основе запроса пользователя, а затем полученная информация подставляется вместе со специальными подсказками, указывающими как модель должна использовать эти данные, в контекст языковой модели для генерации итогового ответа.

Целью данной лабораторной работы является разработка алгоритма реализации метода RAG для вопросно-ответного поиска по онтологии. В качестве базы знаний используется онтология, представленная в виде графа узлов и связей, которая предварительно преобразуется в набор текстовых фрагментов. Для каждого фрагмента вычисляется векторное представление (эмбединг), используемое при поиске релевантной информации.

Генерация ответов осуществляется с помощью предобученной языковой модели `distilbert/distilgpt2`, запускаемой локально в среде Python без использования внешних API. На основе эмбедингов пользовательского запроса производится отбор наиболее близких по смыслу узлов онтологии, после чего соответствующие текстовые фрагменты передаются в модель в составе промпта для генерации ответа.

2. Листинги

Листинг 1 - llm.py

В данном листинге представлена реализация функции загрузки и инициализации языковой модели, используемой для генерации ответов в RAG-системе.

В начале объявляется глобальная переменная `_llm`, предназначенная для хранения экземпляра языковой модели. Это позволяет загружать модель только один раз и повторно использовать её при последующих запросах, что снижает накладные расходы по времени и памяти.

Функция `get_llm()` принимает имя модели в качестве параметра (по умолчанию используется модель `distilbert/distilgpt2`). При первом вызове функции происходит загрузка токенизатора и модели с помощью библиотеки `transformers`. Для этого используются классы `AutoTokenizer` и `AutoModelForCausalLM`, которые автоматически подбирают корректную конфигурацию модели.

После загрузки создаётся объект `pipeline` с задачей `"text-generation"`, который инкапсулирует процесс генерации текста. Модель настраивается для выполнения вычислений на центральном процессоре (параметр `device = -1`), что соответствует требованиям лабораторной работы и позволяет запускать алгоритм без использования графического ускорителя.

При повторных вызовах функции `get_llm()` уже загруженная модель возвращается из глобальной переменной `_llm`, без повторной инициализации.

Таким образом, данный листинг реализует механизм ленивой загрузки языковой модели и обеспечивает её эффективное использование в процессе генерации ответов в RAG-алгоритме.

`truncate_prompt_to_tokens` для предотвращения $N + M > 1024$ (лимит для `distilbert/distilgpt2`)

```
2 usages
def get_llm(model_name="distilbert/distilgpt2"):
    global _llm
    if _llm is None:
        print(f"[LLM] Загружаем модель: {model_name}")
        tokenizer = AutoTokenizer.from_pretrained(model_name)
        model = AutoModelForCausalLM.from_pretrained(model_name)

        _llm = pipeline(
            task="text-generation",
            model=model,
            tokenizer=tokenizer,
            device=-1 # CPU
        )
    return _llm
```

usage

```
def ask_llm(question: str, texts: list) -> str:
    texts_short = [t[:500] for t in texts]

    llm = get_llm()
    tokenizer = llm.tokenizer

    prompt = (
        "Ты — помощник для ответов на вопросы по тексту. "
        "Ответ должен базироваться только на предоставленной информации.\n\n"
        f"Вопрос: {question}\n\n"
        "Текст:\n" + "\n\n".join(texts_short) + "\n\n"
    )
```

```
39     llm = get_llm()
40     out = llm(
41         prompt,
42         max_new_tokens=100,
43         temperature=0.2,
44         do_sample=True,
45         repetition_penalty=1.1
46     )
47
48     # получаем часть с ответом
49     return out[0]["generated_text"]
```

usage

```
51 def truncate_prompt_to_tokens(prompt, tokenizer, max_tokens=900):
52     tokens = tokenizer.encode(prompt, add_special_tokens=False)
53     if len(tokens) > max_tokens:
54         tokens = tokens[:max_tokens]
55     return tokenizer.decode(tokens)
```

Листинг 2 – Загрузка онтологии и извлечение узлов и связей

Загружает онтологию из JSON-файла, извлекает список узлов и связей между ними. Используется на первом этапе RAG для подготовки базы знаний.

```
2 usages
def load_ontology(path):
    with open(path, encoding="utf-8") as f:
        g = json.load(f)

    nodes = g.get("nodes", [])
    edges = find_edges(g)

    print(f"Узлов: {len(nodes)}, связей: {len(edges)}")
    return nodes, edges
```

Листинг 3 – Преобразование узла онтологии в текст

Преобразует структурированное представление узла онтологии (название, описание, тип, связи) в текстовый фрагмент.

Этот текст используется как «документ» в RAG-поиске.

```
2 usages
def node_to_text(node, edges, nodes_by_uri):
    data = node.get("data", {})
    params = data.get("params_values", {})

    uri = data.get("uri")
    label = extract_label(params)
    comment = extract_comment(params)

    lines = []
```

Листинг 4 – Построение индекса онтологии

Формирует векторный индекс онтологии:

- генерирует текстовые представления узлов;
- вычисляет их эмбединги;
- сохраняет эмбединги и метаданные для дальнейшего поиска.

```
def build_ontology_index(nodes, edges):
    nodes_by_uri = {
        n["id"]: n for n in nodes
        if "id" in n
    }

    texts = []
    meta = []

    for node in nodes:
        text = node_to_text(node, edges, nodes_by_uri)
        if not text.strip():
            continue

        texts.append(text)
        meta.append({
            "node_id": node["id"],
            "text": text
        })
```

Листинг 5 – Вычисление текстовых эмбеддингов

Преобразует текстовые фрагменты (узлы онтологии и запрос пользователя) в векторное представление.

Используется как для индексации, так и для семантического поиска.

```
def get_model():
    global _model
    if _model is None:
        _model = SentenceTransformer(
            "sentence-transformers/paraphrase-multilingual-mpnet-base-v2"
        )
    return _model

4 usages
def embed(texts):
    model = get_model()
    return model.encode(texts, convert_to_numpy=True)
```

Листинг 6 – Семантический поиск по онтологии

Выполняет поиск наиболее релевантных узлов онтологии на основе косинусного сходства между эмбеддингом запроса и эмбеддингами узлов.

```
def search(query: str, index, top_k: int = 5):
    query_vec = embed([query])[0]

    sims = cosine_similarity(
        X=[query_vec],
        index["vectors"]
    )[0]

    ranked = sorted(
        zip(sims, index["meta"]),
        key=lambda x: x[0],
        reverse=True
    )

    return ranked[:top_k]
```

Листинг 7 – Основной RAG-конвейер

Реализует полный алгоритм RAG:

1. Поиск N релевантных узлов по запросу пользователя.
2. Генерация промежуточного ответа языковой моделью.
3. Повторный поиск M узлов по эмбеддингу ответа.
4. Генерация финального ответа на основе объединённого контекста.

```

2 usages
def rag_answer(question: str, index, n=5, m=3):
    # Фаза 2
    first = search(question, index, top_k=n)
    texts_n = [item[1]["text"] for item in first]

    print("→ LLM: первый ответ")
    draft_answer = ask_llm(question, texts_n)
    print("✓ Первый ответ получен")

    summary = draft_answer.split("\n")[0][:200]

    # Фаза 3
    second = search(summary, index, top_k=m)
    texts_m = [item[1]["text"] for item in second]

    all_texts = {t for t in texts_n + texts_m}

    print("→ LLM: финальный ответ")
    final_answer = ask_llm(question, list(all_texts))
    print("✓ Финальный ответ получен")

    return final_answer

```

Листинг 8 – Извлечение текстовых атрибутов RDF

Обеспечивает корректное извлечение названий и описаний узлов онтологии, что повышает качество текстового представления и семантического поиска.

```

3 usages
def extract_label(params_values: dict):
    labels = params_values.get(
        "http://www.w3.org/2000/01/rdf-schema#label", []
    )
    for l in labels:
        if l.endswith("@ru"):
            return l.replace("@ru", "")
    return labels[0].replace("@en", "") if labels else None

2 usages
def extract_comment(params_values: dict):
    return params_values.get(
        "http://www.w3.org/2000/01/rdf-schema#comment"
    )

```

3. Источники информации

1. **Hugging Face Documentation**

Transformers Library

<https://huggingface.co/docs/transformers>

Официальная документация по использованию моделей генерации текста и эмбедингов в Python.

2. **Sentence-Transformers Documentation**

<https://www.sbert.net>

Описание моделей и методов для построения текстовых эмбедингов и семантического поиска.

3. **OpenAI Community – GPT-2 / DistilGPT-2**

<https://huggingface.co/distilbert/distilgpt2>

— Описание используемой языковой модели генерации текста.